

Aula 5: Diagnóstico dos Resíduos

Silvio Costa

2025-09-16

Plano

- Analisar o ajuste de um modelo aos dados
 - Testes de Normalidade
 - Testes de Autocorrelação serial
 - Teste de Heteroscedasticidade Condicional
 - Teste de erro de especificação
-

1. Diagnóstico dos Resíduos

- Uma das hipóteses fundamentais dos modelos ARMA(p,q) é que o termo de erro é um **ruído branco**.
 - Vimos que o teste de Ljung-Box pode nos ajudar a fazer inferência sobre as autocorrelações da série de resíduos. Se rejeitamos H_0 no teste de Ljung-Box, significa que alguma autocorrelação não é nula, ou seja, há informação relevante nos resíduos (dependência temporal) que pode ser melhor capturada pelo econometrista com outro modelo.
 - Então, o objetivo é buscar um modelo em que o correlograma dos resíduos não apresentem qualquer memória e que não descartemos a hipótese nula do teste de Ljung-Box.
 - Na prática, olhamos as funções FAC e FACP para tentar descobrir qual autocorrelação deveria ser melhor modelada. Fazemos isso desde o começo, olhando o correlograma da série e dos resíduos dos modelos testados.
-

2. Teste de Normalidade

- A intuição é verificar se os resíduos podem ser considerados como normalmente distribuídos.
- A inspeção visual da distribuição dos resíduos quanto à sua média e simetria será um elemento importante.
- Nota: Dependendo da amostra, valores extremos (outliers) podem dificultar muito que o histograma dos dados se comporte como o histograma de uma distribuição normal.
- Para evitar o erro de supervalorizar eventos extremos, usamos uma função suavizadora (Kernel) para o histograma dos resíduos.
- Função Kernel

$$\hat{f}_h = \frac{\sum_{t=1}^T K\left(\frac{\epsilon - \hat{\epsilon}_t}{h}\right)}{Th}$$

onde h é a largura da janela de suavização; $K(\cdot)$ é a função Kernel, tipicamente usamos uma FDP simétrica com média zero; e $\hat{\epsilon}_t^s = \frac{\epsilon_t - \bar{\epsilon}}{\hat{\sigma}}$ é o resíduo padronizado.

- A ponderação serve para suavizar os pontos observados, colocando menos peso nos erros mais distantes.
- Ver exemplos de função Kernel em Bueno, página 81.

```
#install.packages("ipeadata")
#install.packages("xts")

library(ipeadata)

library(xts)

## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric

ibov = ipeadata("GM366_IBVSP366") # ibovespa

dados.ibovespa = xts( ibov$value, order.by=ibov$date )

dados.ret_ibovespa = log(dados.ibovespa/lag(dados.ibovespa)) * 100
dados.ret_ibovespa <- na.omit(dados.ret_ibovespa)
dados.ret_ibovespa <- as.ts(dados.ret_ibovespa)

# Estimar a densidade kernel
densidade <- density(dados.ret_ibovespa, kernel = "gaussian")

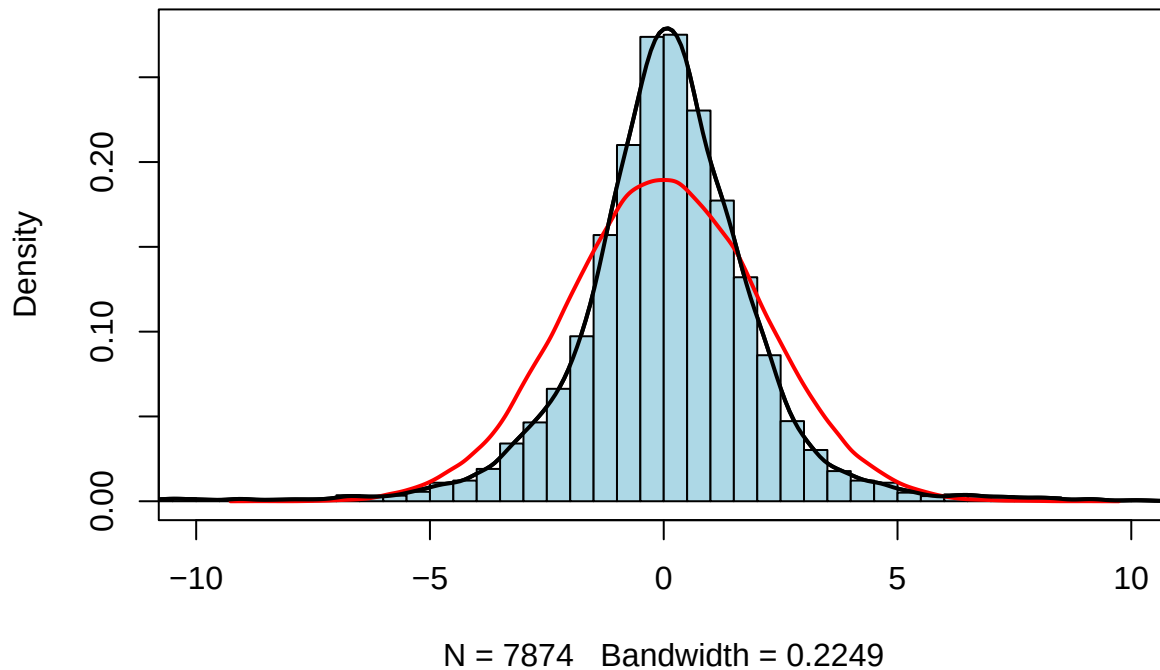
# Plotar a densidade kernel
plot(densidade, main = "Estimativa de Densidade Kernel", xlim=c(-10, 10), col = "black", lwd=2)

hist(dados.ret_ibovespa, freq = FALSE, add = TRUE, breaks = 100, col = "lightblue")

sigma_emp = sd(dados.ret_ibovespa)
lines(density(rnorm(100000,0, sigma_emp)), col = "red", lwd=2)

lines(densidade, col = "black", lwd=2)
```

Estimativa de Densidade Kernel



3. Teste de Jarque-Bera

Outra forma de verificar a normalidade dos resíduos é testar se os momentos da série estimada são iguais aos da normal.

A hipótese nula é que a (Assimetria = 0) E (Curtose=3). Mais formalmente:

$$H_0: E[\epsilon_t^s]^3 = 0 \quad \wedge \quad E[\epsilon_t^s]^4 = 3$$

$$H_A: E[\epsilon_t^s]^3 \neq 0 \quad \vee \quad E[\epsilon_t^s]^4 \neq 3$$

A estatística do teste de Jarque-Bera é dada por:

$$JB = \frac{T}{6} \left(\frac{\sum_{t=1}^T (\epsilon_t^s)^3}{T} \right)^2 + \frac{T}{24} \left(\frac{\sum_{t=1}^T (\epsilon_t^s)^4}{T} - 3 \right) \sim \chi^2(2)$$

O teste de hipótese é: se rejeitar H_0 significa não normalidade dos resíduos. Lembre-se de que não rejeitar H_0 não significa normalidade!

Vamos realizar o teste de Jarque-Bera nos retornos do Ibovespa. O que isso significa?

```
#install.packages("moments")
```

```
library(moments)
```

```
# Calculando os momentos
```

```
mean_x <- mean(dados.ret_ibovespa)
```

```
# Primeiro momento: média
```

```
var_x <- var(dados.ret_ibovespa)
```

```
# Segundo momento: variância
```

```
skew_x <- skewness(dados.ret_ibovespa)
```

```
# Terceiro momento: assimetria
```

```
kurt_x <- kurtosis(dados.ret_ibovespa)
```

```
# Quarto momento: curtose
```

```
# Criando a tabela com os momentos
momentos <- data.frame(
  Momento = c("Média", "Variância", "Assimetria", "Curtose"),
  Valor = c(mean_x, var_x, skew_x, kurt_x)
)
```

```
# Exibindo a tabela
print(momentos)
```

```
##      Momento      Valor
## 1      Média  0.07536207
## 2  Variância  4.34405060
## 3 Assimetria  0.32593147
## 4      Curtose 16.02023238
```

```
#install.packages("tseries")
```

```
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
## as.zoo.data.frame zoo
```

```
# Realizando o teste de Jarque-Bera
jb_test <- jarque.bera.test(dados.ret_ibovespa)
```

```
# Exibindo os resultados
print(jb_test)
```

```
##
## Jarque Bera Test
##
## data: dados.ret_ibovespa
## X-squared = 55758, df = 2, p-value < 2.2e-16
```

Teste de normalidade de Shapiro-Wilk

```
# Realizando o teste de Shapiro-Wilk
sw_test <- shapiro.test(dados.ret_ibovespa[1:5000])
```

```
# Exibindo os resultados
print(sw_test)
```

```
##
## Shapiro-Wilk normality test
##
## data: dados.ret_ibovespa[1:5000]
## W = 0.91818, p-value < 2.2e-16
```

4. Teste LM

- O teste LM para autocorrelação dos resíduos também é conhecido como teste de Breusch-Godfrey.
- Suponha a regressão de termos defasados dos resíduos:

$$\hat{\varepsilon}_t = \beta_1 \varepsilon_{t-1} + \beta_2 \varepsilon_{t-2} + \dots + \beta_h \varepsilon_{t-h} + u_t$$

- Se os resíduos forem autocorrelacionados, os betas estimados devem ser diferentes de zero. A hipótese nula é que todos os betas são iguais a zero, ou seja, não há autocorrelação.
- A estatística do teste é $LM_h = T * R^2 \sim \chi^2(h)$

```
# Instalando e carregando a biblioteca 'lmtest'
#install.packages("lmtest")

#install.packages("forecast")

library(lmtest)

library(forecast)

# meu modelo
modelo_ar1 <- arima(dados.ret_ibovespa, order= c(1,0,0))

residuos <- residuals(modelo_ar1)

# Ajuste um modelo linear com os resíduos defasados
mod_resid_lm <- lm(residuos ~ ., data = residuos)
# summary(mod_resid_lm)

# Realizando o teste de Breusch-Godfrey para autocorrelação de ordem até 12
bg_test <- bgtest(mod_resid_lm, order = 12, type = "Chisq")
# summary(bg_test)

# Exibindo os resultados
print(bg_test)

##
## Breusch-Godfrey test for serial correlation of order up to 12
##
## data: mod_resid_lm
## LM test = 25.216, df = 12, p-value = 0.01383
```

5. Teste ARCH-LM

- O teste ARCH-LM serve para identificar sinais de heteroscedasticidade condicional nos resíduos.
- Esse tipo de heteroscedasticidade ocorre quando a variância dos erros de um modelo depende dos valores passados dos erros, ou seja, há uma autocorrelação na variância dos erros.
- Para implementar o teste, usamos os resíduos estimados do modelo.
- Fazemos uma regressão auxiliar dos dos resíduos ao quadrado sobre os valores defasados dos resíduos ao quadrado. A equação da regressão auxiliar é assim:

$$\epsilon_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \alpha_2 \epsilon_{t-2}^2 + \dots + \alpha_q \epsilon_{t-q}^2 + u_t$$

- Se houver correlação entre a variância ao longo do tempo, os coeficientes α estimados serão diferentes de zero.
- A hipótese nula do teste é que todos os coeficientes α 's são idênticos e nulos.
- A estatística LM é calculada a partir da regressão auxiliar:

$$LM = T \cdot R^2$$

sendo T é o número de observações e R^2 é o coeficiente de determinação da regressão auxiliar.

- Se o valor p do teste ARCH-LM for menor que o nível de significância escolhido (por exemplo, 0.05), rejeita-se a hipótese nula de homocedasticidade e conclui-se que há evidência de heterocedasticidade condicional nos resíduos do modelo.

```
# Regressão auxiliar
modelo_aux <- lm(I(residuos^2) ~ lag(I(residuos^2), 1) + lag(I(residuos^2), 2) + lag(I(residuos^2), 3))
resultado_arch_lm <- bgtest(modelo_aux, order = 3)

# Resultado do teste
print(resultado_arch_lm)

##
## Breusch-Godfrey test for serial correlation of order up to 3
##
## data:  modelo_aux
## LM test = 72.486, df = 3, p-value = 1.253e-15
```

6. Teste RESET

- O teste RESET (*Regression Equation Specification Error Test*) foi proposto por Ramsey em 1969 e é utilizado para verificar se há erros de especificação na forma funcional de um modelo de regressão.
- Testa a presença de não linearidades na série. Lembre-se que estamos tentando explicar a série com combinações lineares de componentes AR e MA.
- A intuição é que se os resíduos do modelo forem independentes, não devem ser correlacionados com nenhuma outra variável, principalmente os regressores defasados que já estão no modelo.
- Seja o modelo proposto $y_t = x_t' \beta + \varepsilon_t$. Estimamos os coeficientes do modelo $\hat{\beta}$ e coletamos os resíduos estimados $\hat{\varepsilon}_t$. Em seguida, fazemos uma regressão auxiliar dos resíduos estimados contra os valores previstos de $\hat{y}_t = x_t' \hat{\beta}$ e componentes adicionais que são potências de $\hat{y}_t$. Ou seja:

$$\hat{\varepsilon}_t = x_t' \beta + \psi_2 \hat{y}_t^2 + \psi_3 \hat{y}_t^3 + \dots + \psi_h \hat{y}_t^h + v_t$$

- A hipótese nula **H0**: o modelo original está corretamente especificado e os coeficientes das funções não-lineares $\psi_2 = \psi_3 = \dots = \psi_h = 0$ são nulos. Não há informação adicional nos elementos não lineares de $\hat{y}_t$. A hipótese alternativa é que modelo original está mal especificado (algum coeficiente das funções não-lineares não é igual a zero).
- A estatística do teste segue uma distribuição F para verificar a significância dos coeficientes das funções não-lineares. Se a estatística F for significativa, há evidências de que o modelo original está mal especificado.

$$RESET_h = \frac{\frac{SSR_1 - SSR_0}{h-1}}{\frac{SSR_0}{T-K}} \sim F(h-1, T-K)$$

- Na prática, testamos h igual a 2 ou 3 como suficientes para detectar potenciais não-linearidades.

```
library(lmtest)

# Realize o teste RESET
reset_test <- resettest(mod_resid_lm, power = 2:3, type="fitted")
print(reset_test)

##
## RESET test
##
## data:  mod_resid_lm
## RESET = 8.342, df1 = 2, df2 = 7870, p-value = 0.0002404
```
